

Lab 1

Lab 1 Problem Statement

Lab Title: Learn to implement linked list data structure

Description:

Write a C++ program to create a Linked List that will store and display the data of faculty members of a university. The user will enter the data of 5 faculty members (name and age). On the basis of age, the post of the faculty member will be decided and then this data will be added into the linked list. Each node of the linked list will contain object of Faculty class in data part and next pointer.

Age Criteria for Faculty member's post:

- Lecturer (if age $\geq$ 25 and age $\leq$ 35)
- Assistant Professor (if age $\geq$ 36 && age $\leq$ 45)
- Professor (if age $\geq$ 46 && age $\leq$ 60)

Structure of Classes:

The program should contain the following classes:

1. Faculty
2. Node
3. List

Faculty Class:

Data Members:

Private variables (string name, int age, string post)

Functions:

Setter and Getter functions

Node Class:

Data Members:

Private variables (Faculty object, Node * nextNode)

Functions:

Faculty get()

void set(Faculty object)

Node * getNext()

void setNext(Node * nextNode)

List Class:

Data Members:

Private variables (int size, Node * headNode, Node * currentNode)

Member Functions:

List();

void add (Faculty addObject); (To Add the data of faculty member into the linked list node)

Faculty get();

bool next();

friend void traverse(List list); (To visit each node and display the data of a faculty members stored in that node)

Solution :

```
#include <iostream>
```

```
using namespace std;
```

```
class Faculty{
```

```
    string name;
```

```
string post;

int age;

public:

    void setName(string name){

        this->name= name;

    }

    void setPost(string post){

        this->post= post;

    }

    void setAge(int age){

        this->age=age;

    }

    string getName(){

        return name;

    }

    string getPost(){

        return post;

    }

    int getAge(){

        return age;

    }
```

```
};  
  
class Node {  
    public:  
        Faculty get() {  
            return object;  
        }  
        void set(Faculty object){  
            this->object.setName(object.getName());  
            this->object.setPost(object.getPost());  
            this->object.setAge(object.getAge());  
        }  
        Node * getNext(){  
            return nextNode;  
        }  
        void setNext(Node * nextNode){  
            this->nextNode = nextNode;  
        }  
    private:  
        Faculty object;  
        Node * nextNode;  
};
```

```
/* The List class */  
  
class List {  
  
public:  
  
    List();  
  
    void add (Faculty addObject);  
  
    Faculty get();  
  
    bool next();  
  
    friend void traverse(List list);  
  
private:  
  
    int size;  
  
    Node * headNode;  
  
    Node * currentNode;  
  
};  
  
/* Constructor */  
  
List::List() {  
  
    headNode = new Node();  
  
    headNode->setNext(NULL);  
  
    currentNode = NULL;  
  
    size = 0;  
  
}
```

```
/* add() class method */  
  
void List::add (Faculty addObject) {  
  
    Node * newNode = new Node();  
  
    newNode->set(addObject);  
  
    if( currentNode != NULL )  
  
    {  
  
        newNode->setNext(currentNode->getNext());  
  
        currentNode->setNext( newNode );  
  
        currentNode = newNode;  
  
    }  
  
    else  
  
    {  
  
        newNode->setNext(NULL);  
  
        headNode->setNext(newNode);  
  
        currentNode = newNode;  
  
    }  
  
    size ++;  
  
}
```

```
/* get() class method */  
  
Faculty List::get() {
```

```
if (currentNode != NULL)
return currentNode->get();
}
```

```
/* next() class method */
```

```
bool List::next() {
    if (currentNode == NULL) return false;
    currentNode = currentNode->getNext();
    if (currentNode == NULL || size == 0)
return false;
    else
return true;
}
```

```
/* Friend function to traverse linked list */
```

```
void traverse(List list) {
    Node* savedCurrentNode = list.currentNode;
    list.currentNode = list.headNode;
    cout<<"====Display
Information===="<<endl<<endl;
    for(int i = 1; list.next(); i++)
    {
```

Faculty

```
Faculty member;

member=list.get();

cout << "Name : " << member.getName()<<endl;

cout << "Age : " << member.getAge()<<endl;

cout << "Post : " << member.getPost()<<endl;

}

list.currentNode = savedCurrentNode;

}

int main() {

Faculty member;

string name,post;

List list;

int age;

cout<<"=====Enter Faculty Information===== "<<endl;

for(int a=0;a<5;a++){

    cout<<"Enter Faculty Member Age: "<<endl;

    cin>>age;

    cout<<"Enter Faculty Member Name: "<<endl;

    cin>>name;

    if(age>=25 && age<=35){

        post="Lecturer";
```

```
}  
  
else if(age>=36 && age<=45){  
    post="Assistant Professor";  
}  
  
else if(age>=46 && age<=60){  
    post="Professor";  
}  
  
member.setName(name);  
member.setAge(age);  
member.setPost(post);  
list.add(member);  
}  
  
traverse(list);  
}
```

Lab 2

Lab Title: Learn to implement stack data structure

Objectives: Get the knowledge of implementing stack data structure using linked list and array in C++ programming language.

Tool: Dev C++

Description:

Write C++ program to create two stacks (one using linked list and one using array). Both stacks will store the integer values and display these values in reverse order. In stack implemented through linked list, user can add any number of nodes but in second stack implemented with array, user will enter only 5 numbers.

Structure of Classes:

The program should contain the following classes:

- 1- Node
- 2- Stack_array
- 3- Stack_linkedlist

Node Class

- ➔ Two private variables (int object, Node * nextNode)
- ➔ Functions:
 - num_get()
 - num_set(int object)
 - Node * getNext()
 - void setNext(Node * nextNode)

Stack_array Class

- ➔ Four private variables (int object, int current, int size, int A [5])
- ➔ Functions:
 - int pop_array ()
 - void push_array (int x)
 - int isFull ()
 - int isEmpty ()

Stack_linkedlist Class

- ➔ one private variable (Node *head)
- ➔ Functions:
 - void push(int n);
 - int pop();

Sample Output:

```

Enter Your Choice
1 : Add Stack Element in linked list
2 : Add Stack Element in array
3 : Pop Stack Elements in linked list and display
4 : Pop Stack Elements in array and display
Enter any other number to Close the program
1
Enter the total number you want to entered
3
Please enter the numbers
1
2
3
Numbers added in linked list successfully

Enter Your Choice
1 : Add Stack Element in linked list
2 : Add Stack Element in array
3 : Pop Stack Elements in linked list and display
4 : Pop Stack Elements in array and display
Enter any other number to Close the program
3
The reverse of input in linked list is
3
2
1

Enter Your Choice
1 : Add Stack Element in linked list
2 : Add Stack Element in array
3 : Pop Stack Elements in linked list and display
4 : Pop Stack Elements in array and display
Enter any other number to Close the program
2
You can enter 5 numbers only in array
1
2
3
4
5

Enter Your Choice
1 : Add Stack Element in linked list
2 : Add Stack Element in array
3 : Pop Stack Elements in linked list and display
4 : Pop Stack Elements in array and display
Enter any other number to Close the program
4
The reverse of input in array is
5
4
3
2
1

Enter Your Choice
1 : Add Stack Element in linked list
2 : Add Stack Element in array
3 : Pop Stack Elements in linked list and display
4 : Pop Stack Elements in array and display
Enter any other number to Close the program
6

```

Note: Numbers must be shown in reverse order only by using pop () method

Lab 2 Solution

```
#include <iostream>
```

```
using namespace std;
```

```
class Node{
```

```
private:
```

```
    int object;
```

```
    Node *nextNode;
```

```
public:
```

```
    int num_get() { return object; };
```

```
    void num_set(int object) { this->object = object; };
```

```
    Node * getNext() { return nextNode; };
```

```
    void setNext(Node * nextNode) { this->nextNode = nextNode; };
```

```
};
```

```
class stack_array
```

```
{
```

```
public:
```

```
    Stack_array() { size = 5; current = -1;} //constructor
```

```
    int pop_array(){ return A[current--];} // The pop function
```

```
    void push_array(int x){A[++current] = x;} // The push function
```

```
int isEmpty(){return ( current == -1 );} // Will return true when stack is empty
```

```
int isFull(){ return ( current == size-1);} // Will return true when stack is full
```

```
private:
```

```
int object; // The data element
```

```
int current; // Index of the array
```

```
int size; // max size of the array
```

```
int A[5]; // Array of 5 elements
```

```
};
```

```
class stack_linkedlist{
```

```
private:
```

```
    Node *head;
```

```
public:
```

```
    void push_list(int n);
```

```
    int pop_list();
```

```
};
```

```
// pushing elements into stack
```

```
void stack_linkedlist :: push_list(int n)
```

```
{
```

```
    Node * newNode = new Node();
```

```
newNode->num_set(n);

newNode->setNext(head);

head = newNode;

}

//popping elements from stack and displaying values

int stack_linkedlist ::pop_list()

{

    if(head==NULL){

        cout<<"List is empty!"<<endl;

    }

    int x =head->num_get();

    Node * p = head;

    head=head->getNext();

    delete p;

    cout<<" \n"<< x;

    return x;

}

int main(){

    int n=0; // option

    int k; // total number to be entered in linked list

    int j; // used to push elements in linked list

    int m; // used to push elements in linked list
```

```
stack_linkedlist s;

stack_array sa;

while (n < 5) {

cout << "\n Enter Your Choice \n";

cout << "1 : Add Stack Element in linked list \n";

cout << "2 : Add Stack Element in array \n";

cout << "3 : Pop Stack Elements in linked list and display \n";

cout << "4 : Pop Stack Elements in array and display \n";

cout << "Enter any other number to Close the program \n";


scanf("%d", &n);


switch (n) {

case 1:

cout << "Enter the total number you want to entered \n";

cin>>k;

cout<<"Please enter the numbers \n";

for(int i = 1; i <= k; i++)

{

cin>>j;

s.push_list(j); // push the element at the top

}

cout<<"Numbers added in linked list successfully \n";
```

```
break;

case 2:

if(sa.isFull()) // checking stack is full or not

    cout << "\n Stack is full, can't insert new element";

else {

        cout << "You can enter 5 numbers only in array \n";

    for(int i = 0; i < 5; i++)

        {

            cin >> m;

            sa.push_array(m); // push the element at the top

        }

    }

break;

case 3:

cout << "The reverse of input in linked list is" << "\n";

    for(int i = 1; i <= k; i++)

        {

            s.pop_list();

        }

break;

case 4:

cout << "The reverse of input in array is" << "\n";

    for (int i = 0; i < 5; i++)
```

```
{  
    if(!sa.isEmpty()) // checking stack is empty or not  
        cout << "\n " << sa.pop_array();  
    else  
        cout << "\n Stack is empty, can't pop";  
}  
default :  
    break;  
}  
}  
return 0;  
}
```

Lab 3

Lab Title: Learn to implement queue data structure

Objectives: Get the knowledge of implementing queue data structure using linked list in C++ programming language.

Tool: Dev C++

Description:

Write a C++ program to create a queue (using linked list), Your program should meet the following requirements:

1. The queue will store only even numbers and just display those numbers without removing them.
2. Then you have to delete the first even element from the queue and show the remaining numbers of queue.

Structure of Classes:

The program should contain the following classes:

- 4- Node
- 5- Queue

Node Class

- ➔ Two private variables (int object, Node * nextNode)
- ➔ Functions:
 - int get ()
 - void set (int object)
 - Node * getNext()
 - void setNext(Node * nextNode)

Queue Class

- ➔ Two public variables (Node *front, *rear)
- ➔ Functions:
 - void enqueue (int x);
 - int dequeue ();
 - void display ();
 - int isEmpty();

Sample Output:

```
Enter the total numbers you want to enter
5
Enter numbers randomly
11
22
33
44
55
The even numbers are
22
44
After Removing the first Even number
44
```

Solution

//Implementation of Queue using Linked List

```
#include<iostream>
```

```
using namespace std;
```

```
class Node{
```

```
    public:
```

```
    int get() { return object; };
```

```
    void set(int object) { this->object = object; };
```

```
    Node * getNext() { return nextNode; };
```

```
    setNext(Node * nextNode) { this->nextNode = nextNode; };
```

```
    int object;
```

```
    Node *nextNode;
```

```
};
```

```
class Queue{
```

```
    public:
```

```
        Node *front,*rear;
```

```
        void enqueue(int x);
```

```
        int dequeue();
```

```
        void display();
```

```
        int isEmpty();
```

```
};
```

```
int Queue::isEmpty()
```

```
{
```

```
    return (front == NULL );
```

```
}
```

```
//Insert an element into Queue
```

```
void Queue::enqueue(int x) {
```

```
    Node* newNode = new Node();
```

```
    newNode->set(x);
```

```
    newNode->setNext(NULL);
```

```
//for first node
```

```
    if(front==NULL){
```

```
        front=rear=newNode;
```

```
    }
```

```
    else{
```

```
        rear->setNext(newNode);
```

```
        rear=newNode;
    }
}

void Queue::display(){
    if(isEmpty()) {
        cout<<"Queue is empty."<<endl;
        return;
    }
    Node *temp=front;
    //will check until NULL is not found
    while(temp != NULL){
        cout<<temp->object<<" \n";
        temp=temp->nextNode;
    }
}

//Removing elements from Queue
int Queue :: dequeue()
{
    int x = front->get();
    Node* p = front;
    front = front->getNext();
    delete p;
    return x;
}
```

```
int main(){
    int n,k;

    Queue Q;

    cout << "Enter the total numbers you want to enter \n";

    cin>>k;

    cout<<"Enter numbers randomly \n";

    for(int i = 1; i <= k; i++)
    {
        cin>>n;

        if(n%2==0)

            Q.enqueue(n);// calling Enqueue for insertion
    }

    cout<<"The even numbers are \n";

    Q.display();

    Q.dequeue();      // calling dequeue

    cout<<"After Removing the first Even number\n";

    Q.display();

    return 0;
}
```

Lab 4

Lab Title: Learn to implement Binary Search Tree Data Structure

Objectives: Get the knowledge of how to construct a Binary Search Tree and perform basic functions on a BST in C++ programming language.

Tool: Dev C++

Description:

Consider the following string array in which different names are stored. The length of each name is different from the others.

➔ `string names [] = {"Saleh", "Ali", "Umar", "Musaddiq", "Rehman", "Hassaan"};`

Write a C++ program to construct a Binary Search Tree which will store the length of each name in its nodes. For example, the length of “Ali” is 3 so 3 will be inserted in BST node. After the construction of BST, you are required to print the values of maximum and minimum nodes of Binary Search Tree.

Your program should meet the following requirements.

1. You will use the same string array mentioned above.
2. Calculate the length of each name and store them in an integer array `treeData`.
3. Construct the Binary Search Tree by inserting the values stored in `treeData`.
4. While inserting the nodes in BST, print each name with its length.
5. The insert method should be able to prevent the entry of duplicate value in BST.
6. The program should be able to print the node having maximum and minimum value.

Structure of Program:

Your program should contain the following:

- 1) A template class `BSTNode` to create nodes of BST
- 2) Non-member functions

template<class Object> BSTNode
- *object: Object - *left : BSTNode - *right : BSTNode
+ BSTNode() + BSTNode(Object *object) + Object *getInfo() + void setInfo(Object *object) + BSTNode *getLeft() + void setLeft(BSTNode *left) + BSTNode *getRight() + void setRight(BSTNode *right) + int isLeaf()

Non member Functions
void insert(BSTNode <int> *root, int *info) int minNode(BSTNode<int>* treeNode) int maxNode(BSTNode<int>* treeNode) main()

Sample Output:

```

Inserting the length of names in BST Nodes one by one
-----
Length of Saleh is: 5
Length of Ali is: 3
Length of Umar is: 4
Length of Musaddiq is: 8
Length of Rehman is: 6
Length of Hassaan is: 7

Value of BST minumum node is: 3
Value of BST maximum node is: 8

```

Solution

```

#include <iostream>
#include <stdlib.h>
#include <conio.h>

```

```

using namespace std;

```

```

template <class Object>
class BSTNode{

```

public:

```
BSTNode() {  
    this->object = NULL;  
    this->left  = NULL;  
    this->right = NULL;  
};
```

```
BSTNode(Object *object) {  
    this->object = object;  
    this->left  = NULL;  
    this->right = NULL;  
};
```

```
Object *getInfo() {  
    return this->object;  
};
```

```
void setInfo(Object *object) {  
    this->object = object;  
};
```

```
BSTNode *getLeft() {  
    return left;  
};
```

```
void setLeft(BSTNode *left) {  
    this->left = left;  
};
```

```
BSTNode *getRight() {
```

```
    return right;

};

void setRight(BSTNode *right) {
    this->right = right;
};

int isLeaf( ) {
    if(this->left == NULL && this->right == NULL)
        return 1;
    return 0;
};

private:
    Object *object;
    BSTNode *left;
    BSTNode *right;

};

void insert(BSTNode <int> *root, int *info) {
    BSTNode <int> *node = new BSTNode <int> (info);
    BSTNode <int> *a, *b;
    a = b = root;
    while(*info != *(a->getInfo()) && b != NULL) {
        a = b;
        if(*info < *(a->getInfo()))
        {
            b = a->getLeft();
        }
    }
}
```

```
    }
else {
    b = a->getRight();
}
}

if(*info == *( a->getInfo() ) ) {
    cout<<endl<<endl<<*info<<"is duplicate in this Binary Search Tree (BST). Can not be inserted .... ";
    getch();
    delete node;
}
else if( *info < *(a->getInfo()) ) {
    a->setLeft(node);
}

else {
    a->setRight(node);
}

cout<<endl;
}

int minNode(BSTNode<int>* treeNode) {
    while (treeNode->getLeft() != NULL) {
        treeNode = treeNode->getLeft();
    }
    return *treeNode->getInfo();
}

int maxNode(BSTNode<int>* treeNode) {
```

```

while (treeNode->getRight() != NULL) {
    treeNode = treeNode->getRight();
}
return *treeNode->getInfo();
}

int main() {
    string names [] = {"Saleh","Ali","Umar","Musaddiq","Rehman","Hassaan"};
    int totalNames = sizeof(names)/sizeof(names[0]);
    BSTNode <int> *NTree = new BSTNode<int>();
    int treeData[totalNames];
    totalNames++;
    for(int a=0;a<totalNames;a++){
        if(a==6){
            treeData[a]=-1;
        }
        else{
            treeData[a]= names[a].length();
        }
    }
    NTree->setInfo( &treeData[0] );
    cout << "Inserting the length of names in BST Nodes one by one" <<endl;
    cout << "-----" << endl;
    cout<<"Length of "<<names[0]<<" is: "<<*NTree->getInfo( )<<endl;
    for(int i = 1; treeData[i] > 0; i++) {
        cout<<"Length of "<<names[i]<<" is: "<<treeData[i];
        insert(NTree, &treeData[i]);
    }
    cout<<"\nValue of BST mininum node is: "<<minNode(NTree)<<endl;

```

```
cout<<"Value of BST maximum node is: "<<maxNode(NTree)<<endl;  
return 0;  
}
```

Lab 5

Lab Title: Learn to implement Binary Search Tree Traversals (In-Order,Pre-Order,Post-Order)

Objectives: Get the knowledge of how to implement Binary Search Tree with character data type and perform different traversal methods on a BST in C++ programming language.

Tool: Dev C++

Description:

Write a C++ program that stores the name of a person and build the binary search tree from the characters of that name. After the construction of the Binary Search Tree, print the characters of name in pre-order, post-order, and in-order traversal.

Your program should meet the following requirements.

1. First, you will enter the length of the name.
2. Enter the name that will be stored in a character array.
3. Construct the Binary Search Tree by inserting the values stored in the character array.
4. The insert method should not enter the duplicate character in binary tree, you can enter the name that contain different characters only.
5. After the construction of Binary tree, the program should be able to print the characters in pre-order, post order, and in-order traversal of the name entered by the user.

Structure of Classes:

The program should contain the following class:

6- TreeNode

TreeNode Class

➔ Three private variables (object* object, TreeNode * left, TreeNode*right)

➔ Functions:

```
TreeNode (Object* object)
Object* getInfo ()
void setInfo (Object* object)
TreeNode* getLeft ()
void setLeft (TreeNode* left)
TreeNode* getRight ()
void setRight (TreeNode* right)
int isLeaf ()
```

Non-Member functions

- ➔ void insert (TreeNode<char>* root, char* info)
- ➔ void inorder (TreeNode<char>* treeNode)
- ➔ void preorder (TreeNode<char>* treeNode)
- ➔ void postorder (TreeNode<char>* treeNode)
- ➔ main ()

Sample Output:

```
Enter the length of a name =6
Enter the name: aleena
attempt to insert duplicate: e
attempt to insert duplicate: a

Inorder: a e l n
preorder: a l e n
postorder: e n l a
-----
```

Solution

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
template <class Object>
```

```
class TreeNode {
```

```
public:
```

```
    // constructors
```

```
    TreeNode()
```

```
{
```

```
this->object = NULL;  
this->left = this->right = NULL;  
};
```

```
TreeNode(Object* object)  
{  
    this->object = object;  
    this->left = this->right = NULL;  
};
```

```
Object* getInfo()  
{  
    return this->object;  
};
```

```
void setInfo(Object* object)  
{  
    this->object = object;  
};
```

```
TreeNode* getLeft()
```

```
{  
    return left;  
};
```

```
void setLeft(TreeNode* left)  
{  
    this->left = left;  
};
```

```
TreeNode* getRight()  
{  
    return right;  
};
```

```
void setRight(TreeNode* right)  
{  
    this->right = right;  
};
```

```
int isLeaf()  
{
```

```
    if (this->left == NULL && this->right == NULL)

        return 1;

    return 0;

};

private:

    Object* object;

    TreeNode* left;

    TreeNode* right;

}; // end class TreeNode

void insert(TreeNode<char>* root, char* info)

{

    TreeNode<char>* node = new TreeNode<char>(info);

    TreeNode<char> *p, *q;

    p = q = root;

    while (*info != *(p->getInfo()) && q != NULL) {

        p = q;

        if (*info < *(p->getInfo()))

            q = p->getLeft();

        else
```

```
        q = p->getRight();
    }

    if (*info == *(p->getInfo())) {
        cout << "attempt to insert duplicate: " << *info << endl;
        delete node;
    }
    else if (*info < *(p->getInfo()))
        p->setLeft(node);
    else
        p->setRight(node);
}

void inorder(TreeNode<char>* treeNode)
{
    if (treeNode != NULL) {
        inorder(treeNode->getLeft());
        cout << *(treeNode->getInfo()) << " ";
        inorder(treeNode->getRight());
    }
}

} //End of inOrder.
```

```
void preorder(TreeNode<char>* treeNode)
{
    if( treeNode != NULL )
    {
        cout << *(treeNode->getInfo())<<" ";
        preorder(treeNode->getLeft());
        preorder(treeNode->getRight());
    }
} //End of preorder

void postorder(TreeNode<char>* treeNode)
{
    if( treeNode != NULL )
    {
        postorder(treeNode->getLeft());
        postorder(treeNode->getRight());
        cout << *(treeNode->getInfo())<<" ";
    }
} // End of post order

int main()
{
```

```
int length;

TreeNode<char>* root = new TreeNode<char>();

cout<<"Enter the length of a name =";

cin>>length;

char* word = new char[length];

int i;

cout<<"Enter the name: ";

for(int i=0;i<length;i++)

    cin>>word[i];

root->setInfo(&word[0]);

for (int i = 1; word[i]!='\0'; i++) {

    insert(root, &word[i]);

}

cout << "\n Inorder: ";

inorder(root);

cout << "\n preorder: ";

preorder( root);

cout << "\n postorder: ";

postorder( root );

return 0;

} // end of main.
```

Lab 6

Lab 6 Problem Statement

Lab Title: Learn to implement function call by value, by reference and by pointer

Description:

Write C++ program to swap the values of two variables of different data types using different function calls. Your program should swap the values using function call by value, reference and pointers. The swapping functions should receive the swapping variables as arguments. The functions should be declared as template functions so that they can entertain any data type.

Structure of Program:

The program should contain the following functions:

4. void swapping_by_value (T obj1, T obj2)
5. void swapping_by_reference (T& obj1, T& obj2)
6. void swapping_by_pointers (T* obj1, T* obj2)

Sample Output:

```
=====Function Call by Value=====
Before Swapping
a = 5
b = 10
After Swapping thorough function call by value no effect on values of A,B
a = 5
b = 10
=====Function Call by Reference=====
Before Swapping
a = 5
b = 10
After Swapping thorough function call by reference
a = 10
b = 5
=====Function Call by Pointers=====
Before Swapping
a = 10
b = 5
After Swapping thorough function call by pointers
a = 5
b = 10
-----
Process exited after 0.3745 seconds with return value 0
Press any key to continue . . .
```

Solution

```
#include <iostream>
```

```
using namespace std;
```

```
template<class T>
```

```
void swapping_by_value(T obj1, T obj2) {
```

```
    T temp = obj1;
```

```
    obj1 = obj2;
```

```
    obj2 = temp;
```

```
}
```

```
template<class T>
```

```
void swapping_by_reference(T& obj1, T& obj2) {
```

```
    T temp = obj1;
```

```
    obj1 = obj2;
```

```
    obj2 = temp;
```

```
}
```

```
template<class T>
```

```
void swapping_by_pointers(T* obj1, T* obj2){
```

```
    T temp= *obj1;
```

```
    *obj1 = *obj2;
```

```
    *obj2 = temp;
```

```
}
```

```
int main(){

    int a=5,b=10;

    cout<<"=====Function Call by Value===== "<<endl<<endl;

    cout<<"Before Swapping"<<endl;

    cout<<"a = "<<a<<endl;

    cout<<"b = "<<b<<endl;

    swapping_by_value(a,b);

    cout<<"After Swapping thorough function call by value no effect on values of A,B"<<endl;

    cout<<"a = "<<a<<endl;

    cout<<"b = "<<b<<endl;

    cout<<"=====Function Call by Reference===== "<<endl<<endl;

    cout<<"Before Swapping"<<endl;

    cout<<"a = "<<a<<endl;

    cout<<"b = "<<b<<endl;

    swapping_by_reference(a,b);

    cout<<"After Swapping thorough function call by refrence"<<endl;

    cout<<"a = "<<a<<endl;

    cout<<"b = "<<b<<endl;

    cout<<"=====Function Call by Pointers===== "<<endl<<endl;

    cout<<"Before Swapping"<<endl;

    cout<<"a = "<<a<<endl;
```

```
cout<<"b = "<<b<<endl;

swapping_by_pointers(&a,&b);

cout<<"After Swapping thorough function call by pointers"<<endl;

cout<<"a = "<<a<<endl;

cout<<"b = "<<b<<endl;

return 0;

}
```

Lab 7

Lectures Covered: 19-22

Lab 7 Problem Statement

Lab Title: Learn to draw AVL Tree.

Objectives: Learn to build/draw AVL tree and understand different types of rotations performed while constructing an AVL tree.

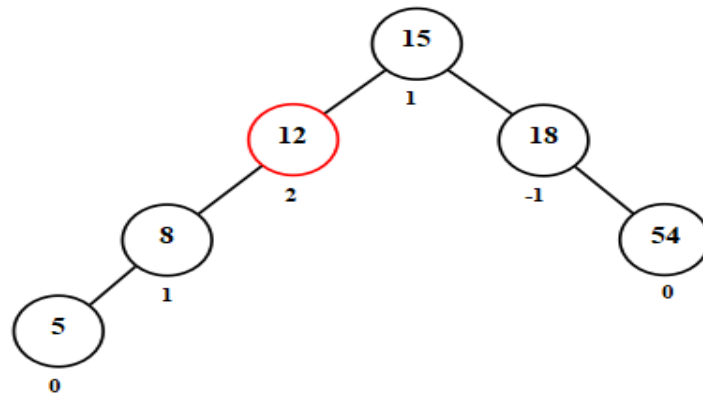
Tool: MS Word, MS Visio or any other online diagram making tool

Description:

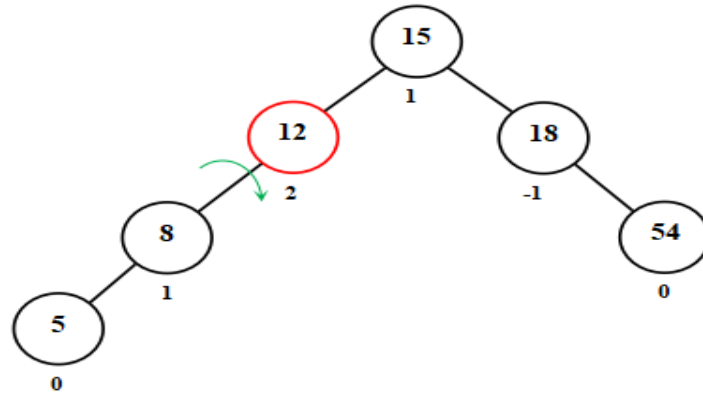
You are required to construct AVL tree from the following data:

15 ,18 ,12, 8, 54, 5, 14, 13, 9, 61, 20, 17, 21

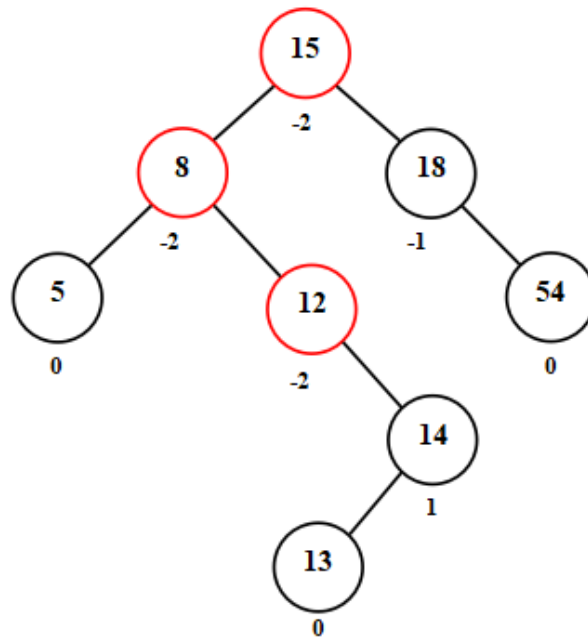
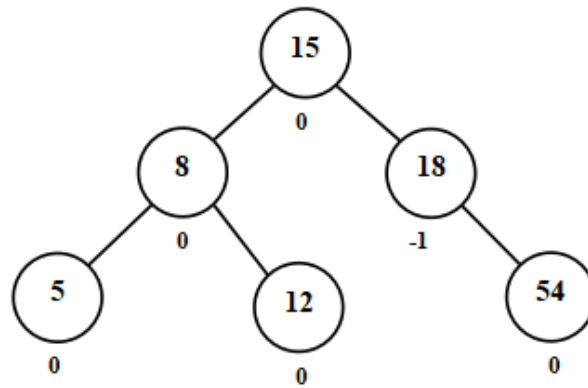
Solution:



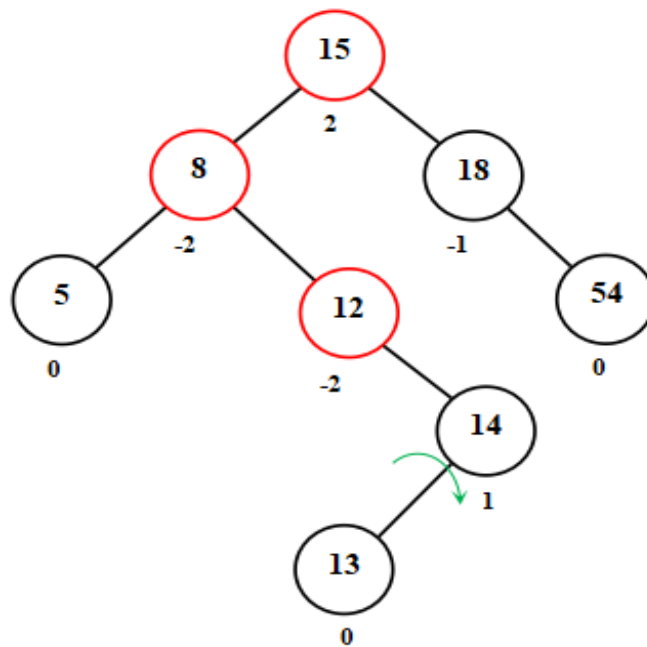
Insert 15, 18, 12, 8, 54 and 5



The balance factor of node 12 is 2, so this node need single right rotation.

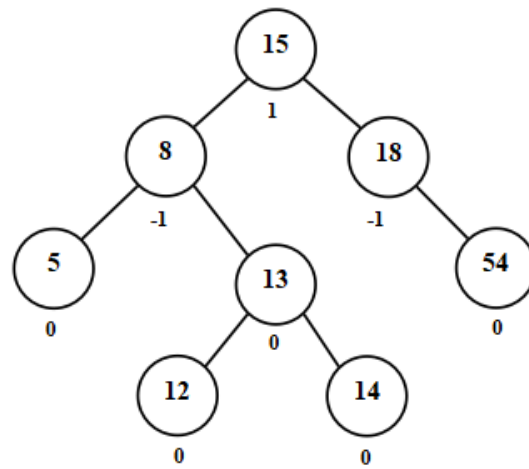
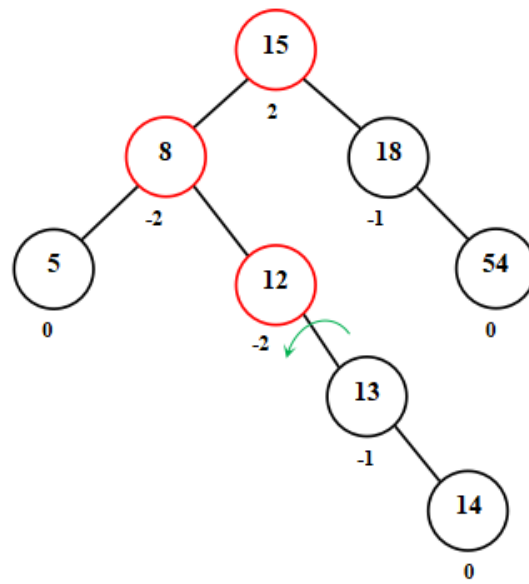


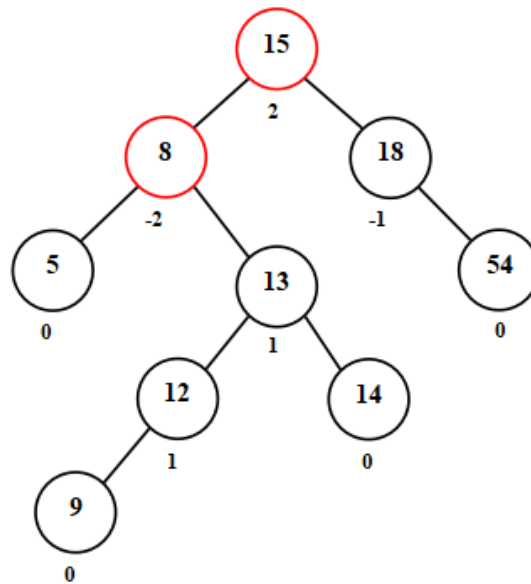
Insert 14 and 13



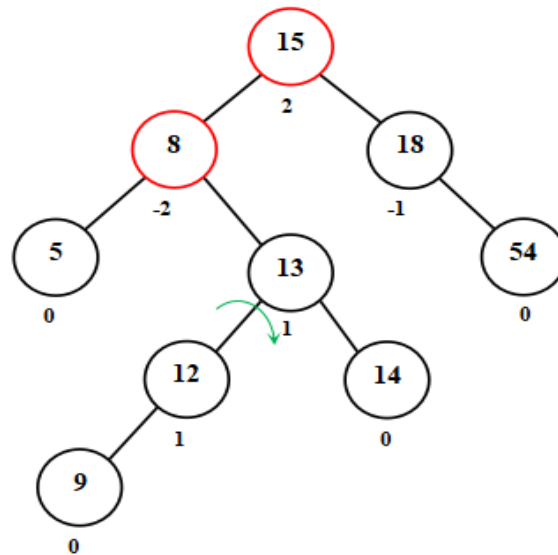
The balance factor of nodes 8 and 12 is -2, also node 15 is 2, so rotation is needed on these nodes. According to rule we will apply rotation on deepest unbalanced node first.

So node 12 needs rotation. Here double right-left rotation is required.



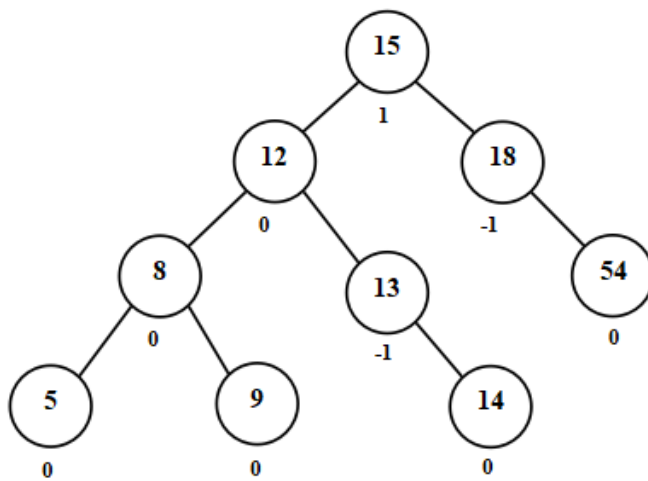
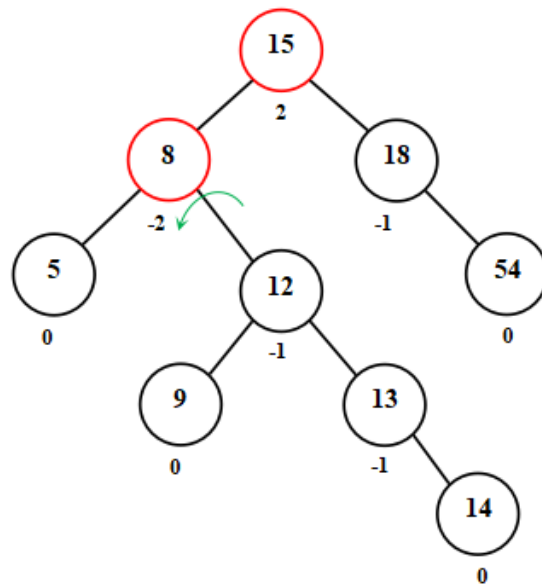


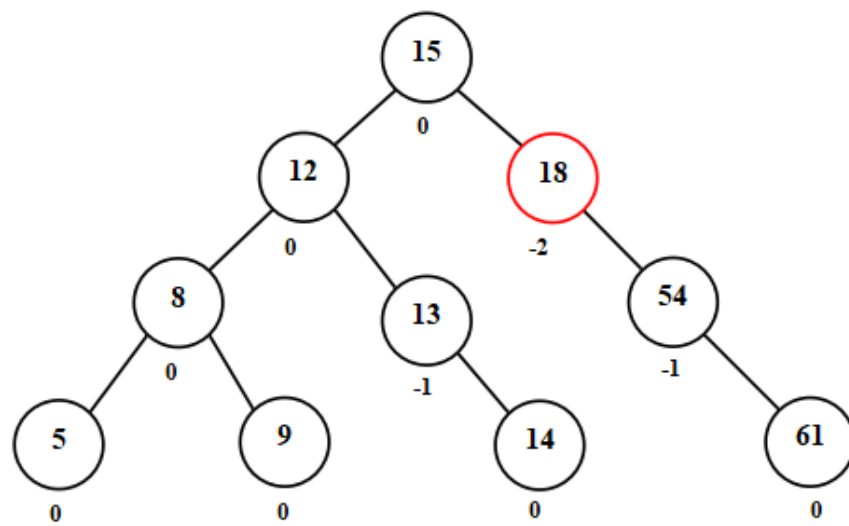
Insert 9



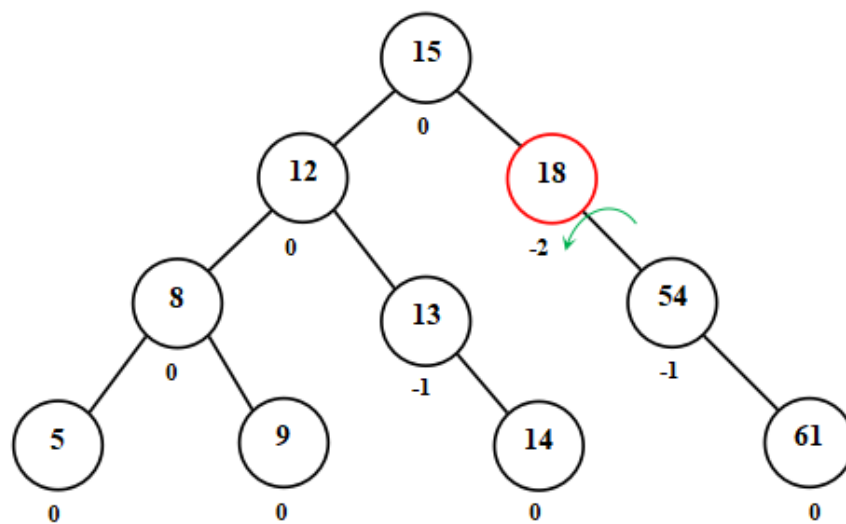
The balance factor of node 8 is -2 and node 15 is 2, so rotation is needed on these nodes. According to rule we will apply rotation on deepest unbalanced node first.

So node 8 needs rotation. Here double right-left rotation is required.

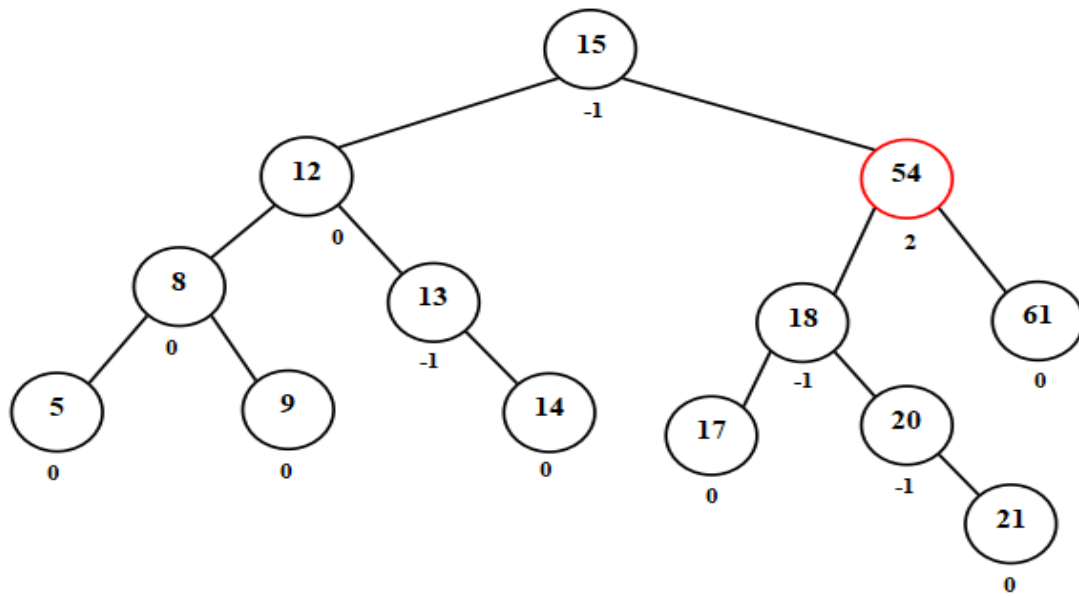
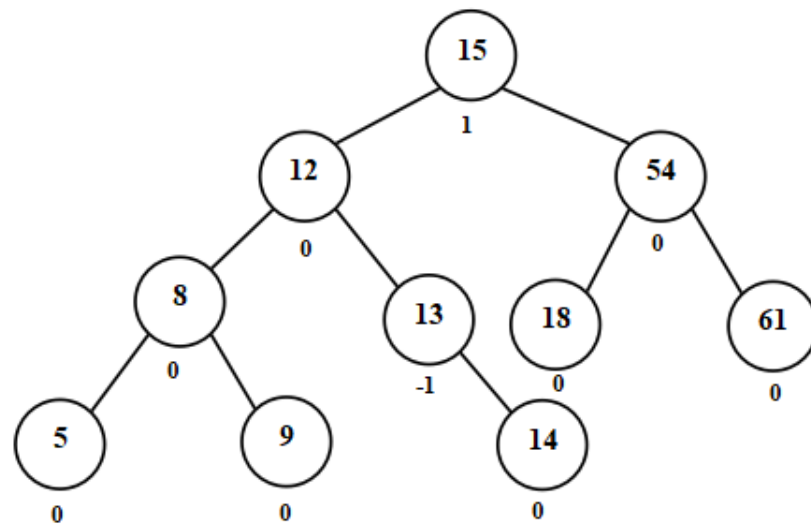




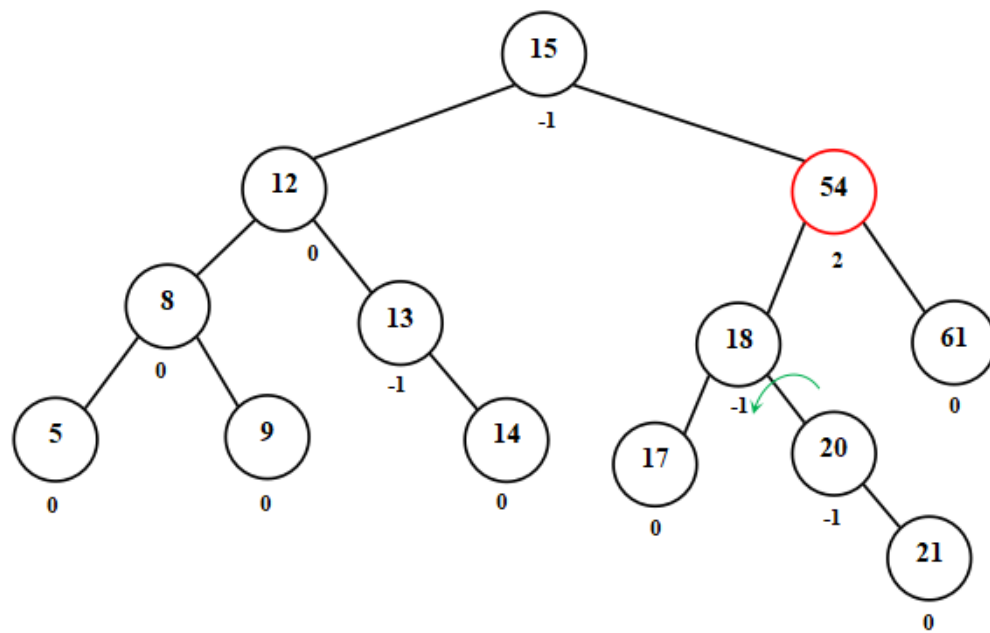
Insert 61



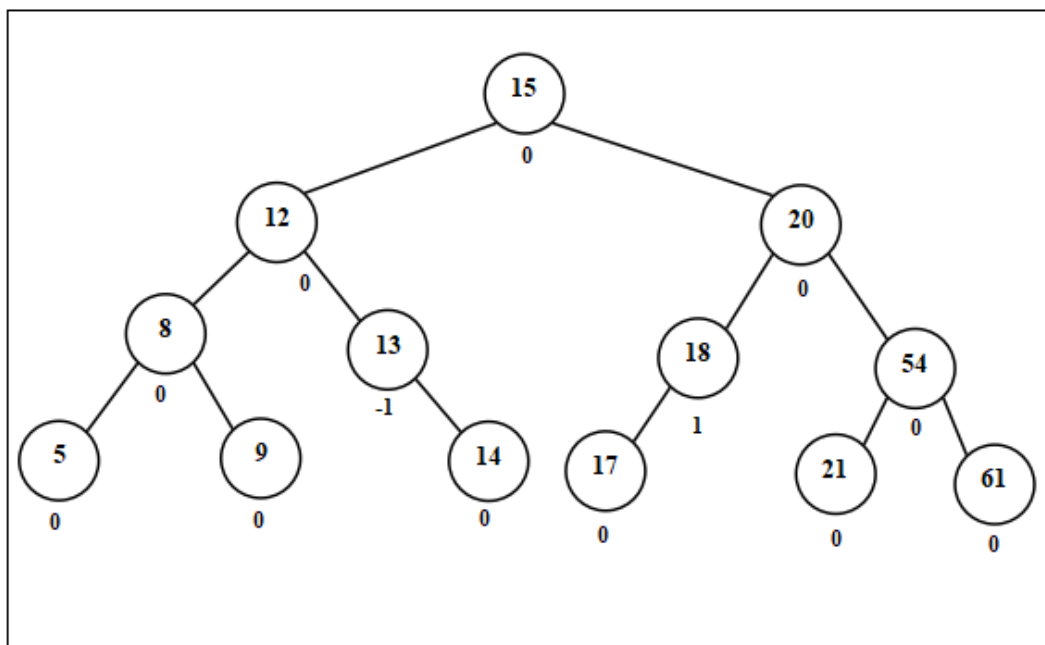
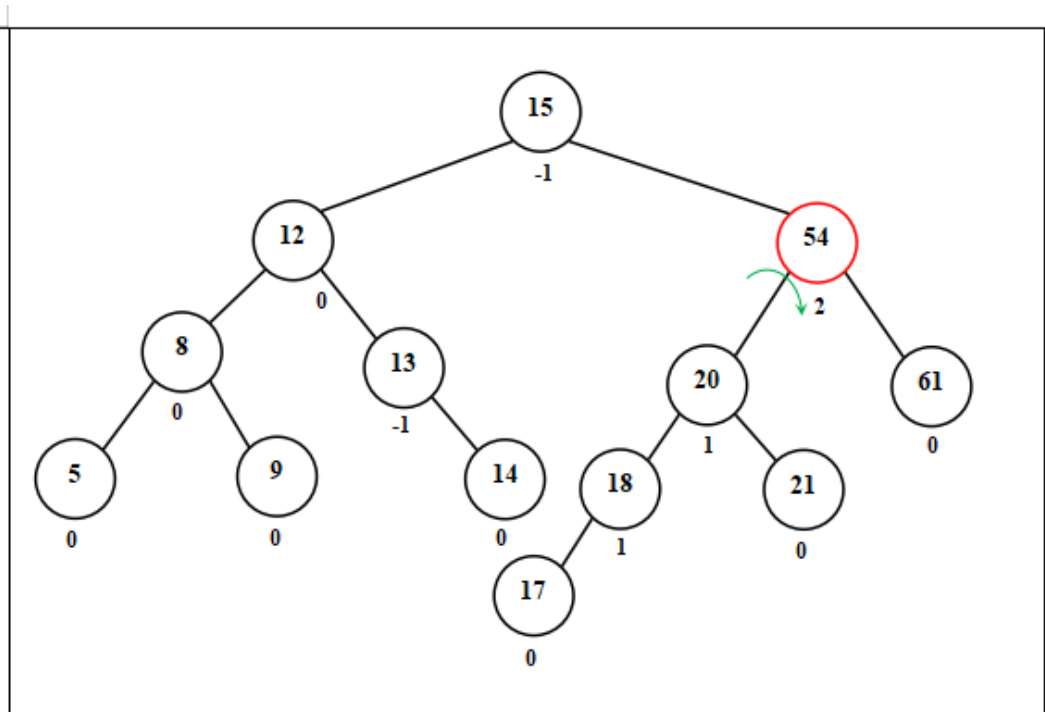
The balance factor of node 18 is -2, so this node needs single left rotation.



Insert 20, 17 and 21



The balance factor of node 54 is 2, so this node needs double left-right rotation.



Lab 8

Lab Title: Learn to draw AVLtree

Lectures Covered: 19-22

Objectives: Learn to build/draw AVL tree and understand different types of rotations

Tool: MS Word, MS Visio

Description:

Build AVL tree from the given Data: 3 5 6 7 9 10 11 21 20 18 19

Note: You have to show only final AVL tree after insertion of each node value.

Lab 9

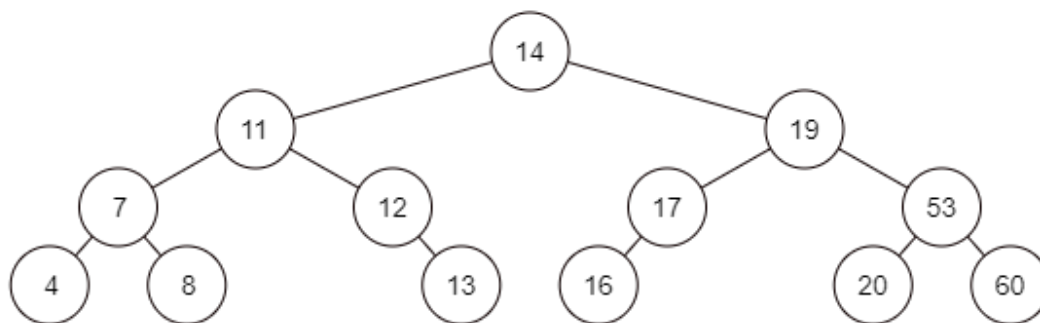
Lab Title: Learn to delete nodes from AVL tree

Objectives: Learn to delete nodes from AVL with the help of rotations

Tool: MS Word, MS Visio

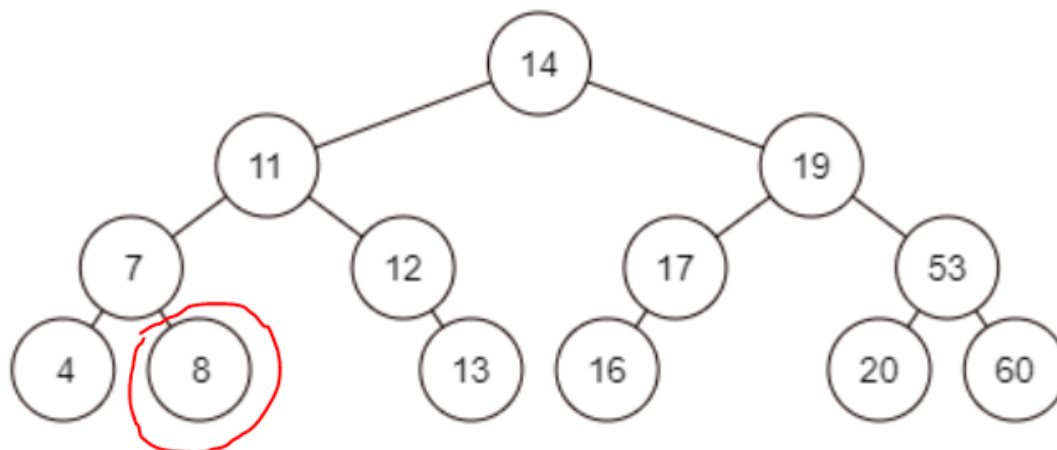
Description:

Delete the node 8,7,11,14,17 from given AVL tree and perform necessary rotation(s) to balance the tree after deletion of each node. Show final tree after each deletion.

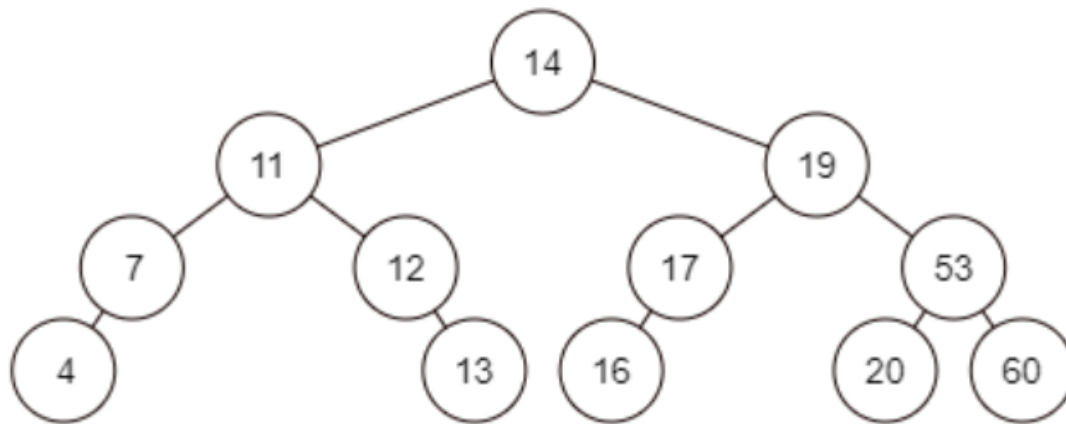


Solution

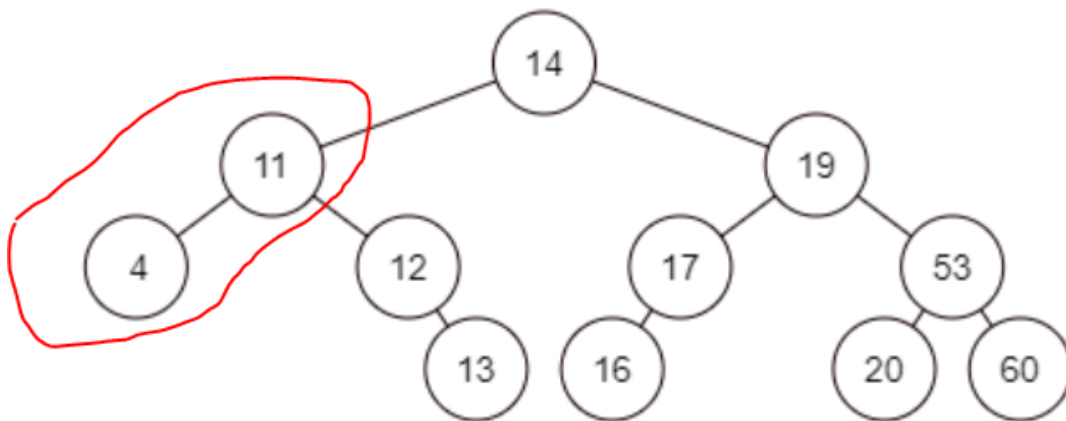
Delete Node 8



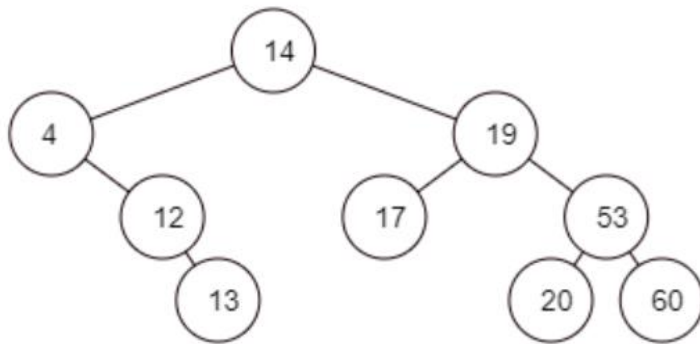
No effect on tree after Node 8 deletion, the tree is balanced. Now we have to delete Node 7



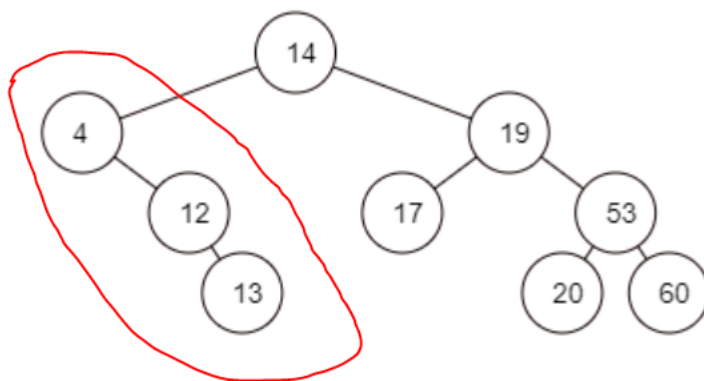
After deleting node 7 the node 4 becomes left child of node 11 and tree is still balanced after deletion



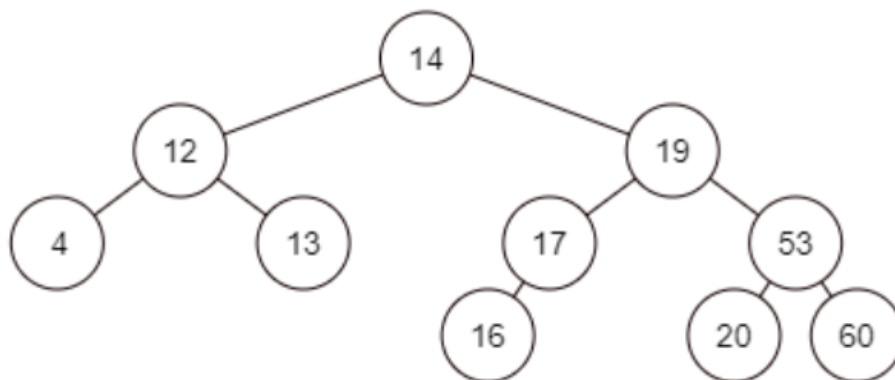
Now we have to delete node 11. The following will be the resultant tree after node 11 Deletion , Node 4 has replaced the node 11



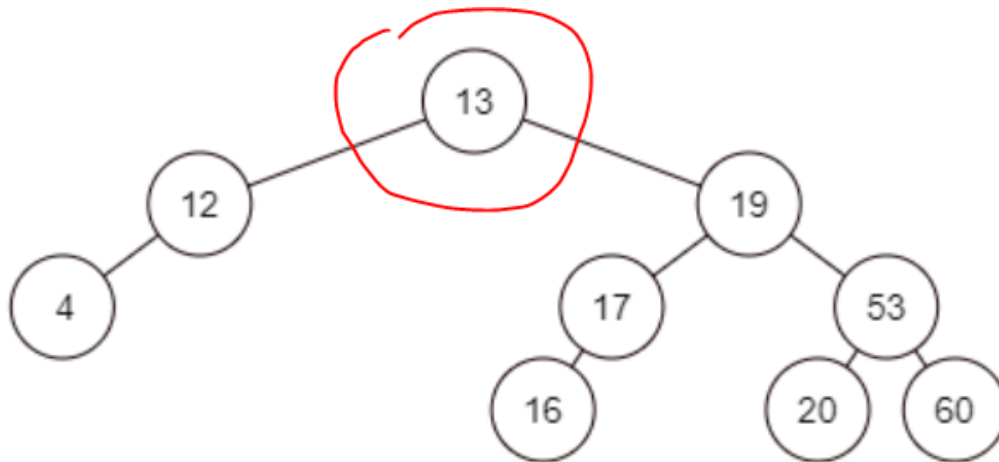
Now if we check the balance after deletion we find that tree is not balanced . Node 4 is unbalanced and hence rotation is required . Here we will apply Left rotation



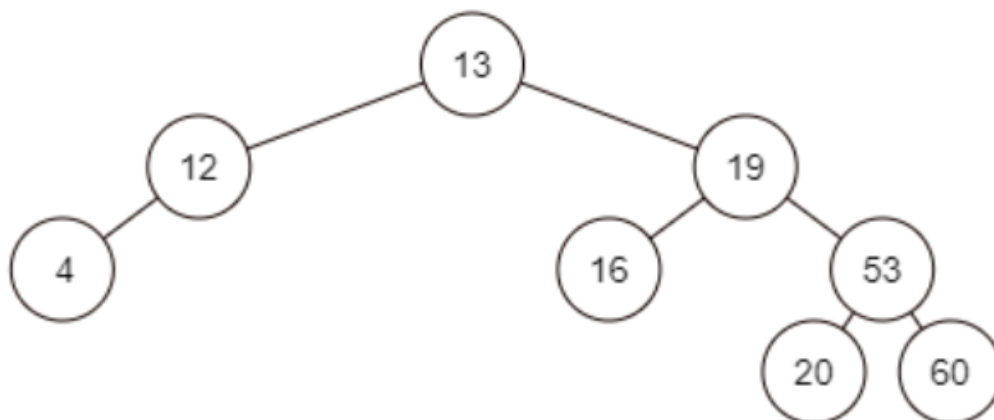
After Applying left rotation following tree is achieved which is balanced



Now we have to delete node 14 . After deleting Node 14 it will be replaced by node node 13. Node 13 will now be the parent node. The tree is balanced after deletion of Node 14



Now we have to make the final deletion i.e. Node 17. After deletion of Node 17 it will be replaced by Node 16. This is our final tree



Lab 10

Lab Title: Learn to build frequency table and Huffman encoding tree

Objectives: Get the knowledge of building frequency table and Huffman encoding tree.

Tool: MS Word, MS Visio

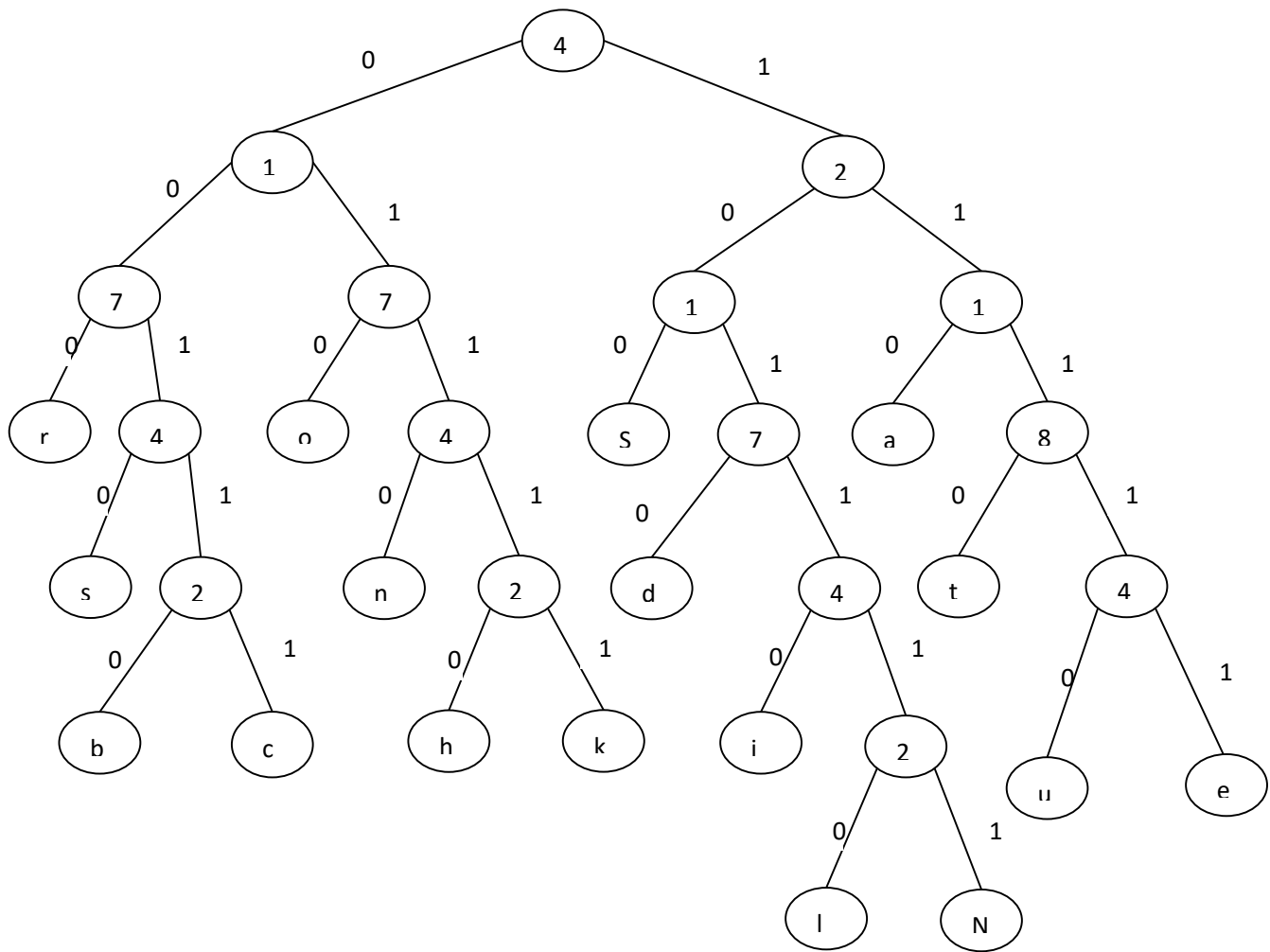
Description:

Consider the message “**the clouds are dark and its about to rain**” and construct frequency table and Huffman encoding tree.

Frequency Table:

Character	Frequency	Character	Frequency
a	5	n	2
b	1	o	3
c	1	r	3
d	3	s	2
e	2	t	4
h	1	u	2
i	2	SP	8
k	1	NL	1
l	1		

Huffman Encoding Tree:



Lab 11

Lab Title: Learn to implement min heap using insert() method

Objectives: Get the knowledge of implementing min heap using insert() method with the help of C++ programming language.

Tool: Dev C++

Description:

Consider the Data: 18, 31, 82, 85, 37, 20, 23, 79, 47, 51, 96, 97, 42, 94, 57, 29 and write the C++ code to construct min heap using insert method.

Solution:

```
#include <iostream>
```

```
using namespace std;
```

```
class Heap {
```

```
    public:
```

```
        Heap (int capacity);
```

```
        bool insert ( const int& x);
```

```
        bool isEmpty ();
```

```
        bool isFull();
```

```
        void traverse();
```

```
    public:
```

```
        int currentSize; // number of elements in heap
```

```
        int * array; // the heap array
```

```
        int capacity;
```

```
};
```

```
Heap::Heap (int capacity) {  
    array = new int[capacity+1];  
    currentSize = 0;  
}  
  
bool Heap::insert ( const int& x) {  
    if(isFull()) {  
        cout<<endl<<"cannot insert Heap is full!!" <<endl;  
        return 0;  
    }  
  
    int hole = ++currentSize;  
    for(/* declaration is above*/ ; hole>1 && x< array[hole/2]; hole /= 2) {  
        array[hole]= array[hole/2];  
    }  
    array[hole] = x;  
}  
  
void Heap::traverse() {  
    for (int i = 1; i <= currentSize; i++)  
        cout<<" "<< array[i] << " ";  
}  
  
bool Heap::isEmpty() {  
    return currentSize == 0;  
}  
  
bool Heap::isFull() {
```

```
        return currentSize==capacity;
    }

    main() {
        int size = 16, arr[size] = {18, 31, 82, 85, 37, 20, 23, 79, 47, 51, 96, 97, 42, 94, 57, 29};
        Heap heap(size);
        for (int i = 0; i < size; i++)
            heap.insert(arr[i]); // insert number into heap one by one

        cout<< " Min Heap using insert method : "<<endl;
        heap.traverse();
    }
```

Lab 12

Lab Title: Learn to implement min heap using buildHeap() method

Objectives: Get the knowledge of implementing min heap using buildHeap() and percolateDown() methods with the help of C++ programming language.

Tool: Dev C++

Description:

Consider the Data: 18, 31, 82, 85, 37, 20, 23, 79, 47, 51, 96, 97, 42, 94, 57, 29 and write the C++ code to construct min heap using buildHeap() method.

```
#include <iostream>
```

```
using namespace std;
```

Solution

```
class Heap {
```

```
    public:
```

```
        Heap (int capacity);
```

```
        bool isEmpty ();
```

```
        bool isFull();
```

```
        void buildHeap(int * anArray, int n);
```

```
        void traverse();
```

```
    public:
```

```
        int currentSize; // number of elements in heap
```

```
        int * array; // the heap array
```

```
        int capacity;
```

```
void percolateDown( int hole );

};

Heap::Heap (int capacity) {
    array = new int[capacity+1];
    currentSize = 0;
}

void Heap::percolateDown(int hole) {
    int child;
    int temp = array[hole];
    for ( /*nothing*/ ; hole * 2 <=currentSize; hole = child) {
        child = hole*2;
        if (child !=currentSize&& array[child+1] < array[child] )
            child++;
        if (array[child]< temp)
            array[hole] = array[child];
        else break;
    }
    array[hole] = temp;
}

void Heap::buildHeap(int * anArray, int n) {
    for (int i = 1; i<= n ; i++ )
```

```
        array[i] = anArray[i-1];

    currentSize = n;

    for (int i =currentSize/2; i>0; i--)

        percolateDown(i);

}

void Heap::traverse() {

    for (int i = 1; i<=currentSize; i++)

        cout<<" "<< array[i] << " ";

}

bool Heap::isEmpty() {

    return currentSize == 0;

}

bool Heap::isFull() {

    return currentSize==capacity;

}

main() {

    int size = 16, arr[size] = {18, 31, 82, 85, 37, 20, 23, 79, 47, 51, 96, 97, 42, 94, 57, 29};

    Heap heap(size);

    heap.buildHeap(&arr[0],size);

    cout<< "\n Min Heap using build method: "<<endl;

    heap.traverse();

}
```

Lab 13

Lab Title: Learn to a build union tree by using union by size method

Objectives: Get the knowledge of building union tree of any given data with the help of union by size method.

Tool: MS Word, MS Visio

Description:

Consider the following set of elements,



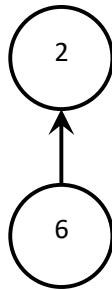
-1	-1	-1	-1	-1	-1	-1	-1
1	2	3	4	5	6	7	8

Apply the following sequence of union commands on the above set of elements and draw the tree by using the union by size method. Show the resultant tree only. Also, update the given array by the union of size method.

- union (2,6)
- union (1,3),
- union (4,2)
- union (1,2)
- union (5,2)

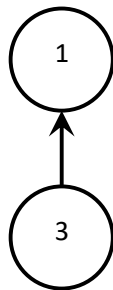
Solution:

- union (2,6)



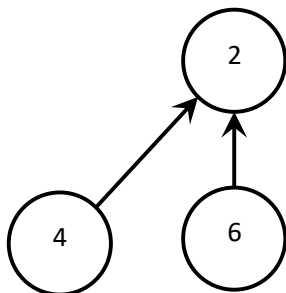
-1	-2	-1	-1	-1	2	-1	-1
1	2	3	4	5	6	7	8

- union (1,3)



-2	-2	1	-1	-1	2	-1	-1
1	2	3	4	5	6	7	8

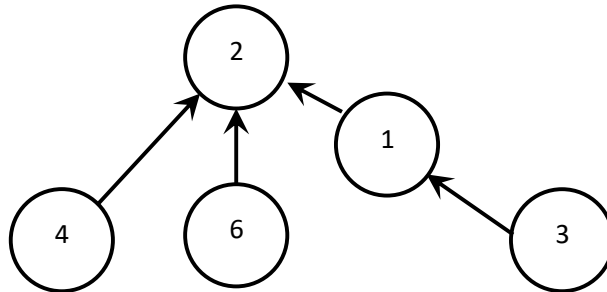
union (4,2)



-2	-3	1	2	-1	2	-1	-1
----	----	---	---	----	---	----	----

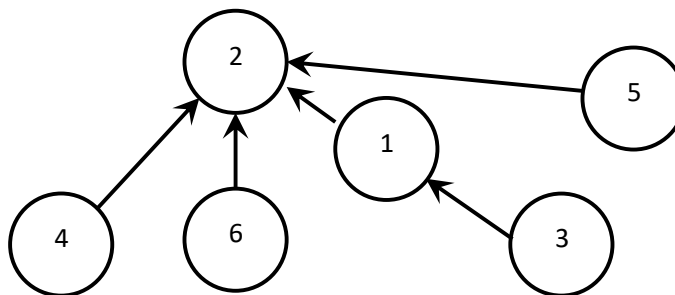
1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

- union (1,2)



2	-5	1	2	-1	2	-1	-1
1	2	3	4	5	6	7	8

- union (5,2)



2	-6	1	2	2	2	-1	-1
1	2	3	4	5	6	7	8

Lab 14

Lab 14 (Lectures Covered 38-40)

Lab Title: Learn to implement binary search algorithm

Description:

Write a program in C++ language to find a number (element) from an array using binary search algorithm. Your program should start search from the first element of array and it should search every third element of the array.

You can use 18, 20, 23, 31, 37, 42, 47, 51, 79, 82, 85, 94, 96, 97 as data of array.

Solution :

```
#include <iostream>

using namespace std;

int isPresent(int arr[], int val, int N){

int low = 0,high = N - 1,mid;

while ( low <= high ){

    mid = ( low + high )/2;

    if (arr[mid] == val)

        return 1;

    else if (arr[mid] < val)

        low = mid + 1;

    else

        high = mid - 1;

}

return 0;

}
```

```
int main() {  
  
    int size = 14, arr[size] = {18, 20, 23, 31, 37, 42, 47, 51, 79, 82, 85, 94, 96, 97};  
  
    for(int i=0;i<=13;i+=3){  
  
        int result = isPresent(arr,arr[i],size);  
  
        if(result == 1)  
  
            cout<<"Element = "<<arr[i]<<" , Index = "<<i<<endl;  
  
        else  
  
            cout<<"Element is not present in array"<<endl;  
  
    }  
  
    return 0;  
  
}
```

Lab 15

Lab Title: Build hash table using linear probing collision resolution technique

Objectives: Learn to build Hash table using linear probing technique to resolve collision

Tool: MS Word

Description:

Insert the values 1, 2, 9, 11,7 into the following table using hash function $(2x + 3) \bmod 8$. You have to apply linear probing technique to resolve collision.

Solution:

0	
1	11
2	7
3	
4	
5	1
6	9
7	2

Lab 16

Lab Title: Learn to sort array using elementary sorting algorithms.

Objectives: Get the knowledge of sorting array using Selection Insertion, and Bubble sort algorithm.

Tool: MS Word

Description:

Consider the data given below as an array and sort by implementing selection, insertion, and bubble sort algorithm.

Selection Sort Algorithm:

You are required to sort the given data in ascending order using selection sort.

8	5	10	3	1
0	1	2	3	4

Solution:

Step 1:

1	5	10	3	8
0	1	2	3	4

Step 2:

1	3	10	5	8
0	1	2	3	4

Step 3:

1	3	5	10	8
0	1	2	3	4

Step 4:

1	3	5	8	10
---	---	---	---	----

0 1 2 3 4

Insertion Sort Algorithm:

You are required to sort the given data in ascending order using insertion sort algorithm

10	4	8	16	11
----	---	---	----	----

Solution:

List: 10 4 8 16 11

Step1: 4 10 8 16 11

Step2: 4 8 10 16 11

Step3: 4 8 10 16 11

Step4: 4 8 10 11 16

Bubble Sort Algorithm:

Suppose, we have been given the following array.

6	1	3	2	9
0	1	2	3	4

You are required to sort the above data in ascending order using Bubble sort..

Solution:**1st iteration:**

Step 1:

6	1	3	2	9
0	1	2	3	4

Step 2:

1	6	3	2	9
0	1	2	3	4

Step 3:

1	3	6	2	9
0	1	2	3	4

Step 4:

1	3	2	6	9
0	1	2	3	4

Step 5:

1	3	2	6	9
0	1	2	3	4

2nd iteration:

Step 1:

1	3	2	6	9
0	1	2	3	4

Step 2:

1	3	2	6	9
0	1	2	3	4

Step 3:

1	2	3	6	9
0	1	2	3	4

Step 4:

1	2	3	6	9
0	1	2	3	4

Step 5:

1	2	3	6	9
---	---	---	---	---

0

1

2

3

4